

DEBACU – MODELO UNIVERSAL PMS + ARQUITECTURA DE INTEGRACIÓN

1.1 Objetivo

Debacu necesita conectarse a distintos PMS sin rediseñar su arquitectura cada vez. Para lograrlo, se define un **modelo universal PMS** que permita normalizar información procedente de proveedores distintos como:

- Tesipro / Ulyses
- Cloudbeds
- Mews
- apaleo
- Oracle OPERA
- otros futuros

El objetivo no es adaptar Debacu a cada PMS, sino construir una **capa de integración desacoplada** donde cada conector traduzca su modelo externo al **modelo canónico interno de Debacu**.

1.2 Principios obligatorios

A. Debacu manda

El modelo interno lo define Debacu.

Ningún PMS puede imponer su estructura al core.

B. Raw + Canonical

Todo dato PMS debe existir en dos niveles:

- **raw**: payload original del PMS
- **canonical**: dato normalizado usable por Debacu

C. Multi-property real

Toda entidad debe quedar trazada por:

- org_id
- property_id
- connection_id

D. Read-only en fase 1

La primera fase solo leerá datos del PMS.

No habrá escritura sobre reservas, huéspedes, inventario ni cargos.

E. Idempotencia

Los conectores deben poder reejecutarse sin duplicar registros ni corromper datos.

F. Normalización explícita

Estados, canales, segmentos y otras clasificaciones deben pasar por tablas de mapping, no por lógica dispersa.

1.3 Alcance funcional v1

El modelo universal v1 cubre:

- propiedades PMS
- room types
- habitaciones
- huéspedes
- reservas
- relación reserva-huéspedes
- estancias
- resumen económico opcional por folio

No cubre aún:

- housekeeping detallado
 - pagos
 - writeback al PMS
 - inventario
 - rate updates
 - POS
 - mantenimiento operativo
-

1.4 Arquitectura conceptual

PMS externo

↓

Conector por proveedor
↓
Ingesta raw
↓
Normalización a modelo Debacu
↓
Tablas canónicas PMS
↓
Módulos Debacu:
- Riesgo
- Revenue
- Analítica
- Alertas
- Dashboard multi-propiedad

1.5 Entidades principales

Integración

- pms_providers
- pms_connections
- pms_property_mappings
- pms_sync_jobs
- pms_raw_records

Modelo canónico

- pms_room_types
- pms_rooms
- pms_guests
- pms_reservations
- pms_reservation_guests
- pms_stays
- pms_folio_summaries

Catálogos de normalización

- pms_status_mappings
 - pms_channel_mappings
 - pms_segment_mappings
-

1.6 Regla de identidad

La PK interna siempre será UUID DebaCu.

Nunca usar como PK interna:

- reservation_id del PMS
- guest_id del PMS
- room_id del PMS

Los IDs externos se guardan como claves de negocio bajo unicidad compuesta:

- (connection_id, external_reservation_id)
 - (connection_id, external_guest_id)
 - (connection_id, external_room_id)
 - (connection_id, external_room_type_id)
-

1.7 Regla de sincronización

Cada conector deberá soportar:

Full sync inicial

Carga completa base.

Incremental sync

Actualización por:

- updated_at
- cursores
- modified_since
- webhooks
- o el mecanismo que permita el PMS

Registro de job

Toda sincronización debe registrar:

- inicio
- fin
- entidad
- resultados

- errores
 - volumen procesado
-

1.8 Reglas para conectores PMS

Todo conector debe:

1. autenticarse desde backend, nunca desde frontend
 2. leer datos del PMS
 3. persistir raw payload
 4. transformar a canonical
 5. aplicar upsert idempotente
 6. registrar job y errores
 7. mantener trazabilidad
 8. no acoplar lógica de UI
 9. no mezclar estructura de proveedor con tablas de negocio Debacu
 10. soportar multi-property si el PMS lo permite
-

1.9 Uso por módulos Debacu

Riesgo

Consumirá sobre todo:

- huéspedes
- reservas
- stays
- histórico por propiedad
- país, documento, canal, recurrencia

Revenue

Consumirá:

- reservas
- estancias
- room types
- habitaciones

- canales
- segmentos
- importes

Dashboard multi-propiedad

Consumirá:

- normalización común de estados
- agregación por org
- detalle por property

1.10 Conclusión

Debacu debe integrar PMS mediante un **modelo canónico único**, no mediante tablas específicas por proveedor.

La arquitectura correcta en v1 es:

- read-only
- raw + canonical
- mappings explícitos
- multi-property
- conectores desacoplados
- sincronización auditable

Eso permite escalar sin convertir el sistema en una colección de parches.

2) SQL INICIAL

MODELO UNIVERSAL PMS – SUPABASE / POSTGRES

Te lo dejo ya en una base razonable para arrancar. Está pensado para PostgreSQL/Supabase.

```
-- =====
-- DEBACU - MODELO UNIVERSAL PMS (V1)
-- Enfoque read-only, multi-property, raw + canonical
-- =====
create extension if not exists pgcrypto;
-- =====
-- 1. ENUMS
```

```

-- =====
do $$
begin
    if not exists (select 1 from pg_type where typname = 'pms_connection_status')
    then
        create type pms_connection_status as enum ('PENDING', 'ACTIVE', 'ERROR',
        'DISCONNECTED');
        end if;
        if not exists (select 1 from pg_type where typname = 'pms_sync_status') then
            create type pms_sync_status as enum ('RUNNING', 'SUCCESS', 'PARTIAL',
            'FAILED');
            end if;
            if not exists (select 1 from pg_type where typname = 'pms_sync_mode') then
                create type pms_sync_mode as enum ('FULL', 'INCREMENTAL');
                end if;
                if not exists (select 1 from pg_type where typname = 'pms_entity_type') then
                    create type pms_entity_type as enum (
                        'PROPERTY',
                        'ROOM_TYPE',
                        'ROOM',
                        'GUEST',
                        'RESERVATION',
                        'STAY',
                        'FOLIO'
                    );
                    end if;
                    if not exists (select 1 from pg_type where typname =
                    'pms_room_operational_status') then
                        create type pms_room_operational_status as enum (
                            'AVAILABLE',
                            'OUT_OF_SERVICE',
                            'DIRTY',
                            'CLEAN',
                            'UNKNOWN'
                        );
                        end if;
                        if not exists (select 1 from pg_type where typname = 'pms_reservation_status')
                        then
                            create type pms_reservation_status as enum (
                                'PENDING',
                                'CONFIRMED',
                                'CHECKED_IN',
                                'CHECKED_OUT',
                                'CANCELLED',
                                'NO_SHOW',
                                'IN_HOUSE',
                                'UNKNOWN'
                            );
                            end if;
                            if not exists (select 1 from pg_type where typname = 'pms_stay_status') then
                                create type pms_stay_status as enum (
                                    'EXPECTED',
                                    'IN_HOUSE',
                                    'CHECKED_OUT',
                                    'CANCELLED',
                                    'NO_SHOW',
                                    'UNKNOWN'
                                );
                                end if;
                                if not exists (select 1 from pg_type where typname = 'pms_guest_role') then
                                    create type pms_guest_role as enum (
                                        'PRIMARY',
                                        'SECONDARY',
                                        'SHARER',

```

```

        'UNKNOWN'
    );
end if;
end $$;
-- =====
-- 2. UPDATED_AT TRIGGER
-- =====
create or replace function set_updated_at()
returns trigger
language plpgsql
as $$
begin
    new.updated_at = now();
    return new;
end;
$$;
-- =====
-- 3. PROVIDERS
-- =====
create table if not exists pms_providers (
    code text primary key,
    name text not null,
    active boolean not null default true,
    auth_type text not null,
    supports_webhooks boolean not null default false,
    supports_multi_property boolean not null default false,
    created_at timestamptz not null default now()
);
insert into pms_providers (code, name, auth_type, supports_webhooks,
supports_multi_property)
values
    ('TESIPRO_ULYSES', 'Tesipro Ulyses PMS', 'CUSTOM', false, true),
    ('CLOUDBEDS', 'Cloudbeds', 'API_KEY', false, true),
    ('MEWS', 'Mews', 'CUSTOM', true, true),
    ('APALEO', 'apaleo', 'OAUTH2', true, true),
    ('OPERA', 'Oracle OPERA / OHIP', 'OAUTH2', true, true)
on conflict (code) do nothing;
-- =====
-- 4. CONNECTIONS
-- =====
create table if not exists pms_connections (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid null,
    provider_code text not null references pms_providers(code),
    environment text not null check (environment in ('sandbox', 'production')),
    external_account_id text null,
    external_group_id text null,
    auth_mode text not null,
    status pms_connection_status not null default 'PENDING',
    last_sync_at timestamptz null,
    last_success_sync_at timestamptz null,
    last_error_at timestamptz null,
    last_error_message text null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);
create index if not exists idx_pms_connections_org on pms_connections(org_id);
create index if not exists idx_pms_connections_property on
pms_connections(property_id);
create index if not exists idx_pms_connections_provider on
pms_connections(provider_code);
drop trigger if exists trg_pms_connections_updated_at on pms_connections;
create trigger trg_pms_connections_updated_at

```



```

before update on pms_connections
for each row execute function set_updated_at();
-- =====
-- 5. PROPERTY MAPPINGS
-- =====
create table if not exists pms_property_mappings (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid not null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    external_property_id text not null,
    external_property_code text null,
    external_property_name text null,
    is_primary boolean not null default false,
    created_at timestamptz not null default now(),
    unique (connection_id, external_property_id)
);
create index if not exists idx_pms_property_mappings_org on
pms_property_mappings(org_id);
create index if not exists idx_pms_property_mappings_property on
pms_property_mappings(property_id);
-- =====
-- 6. SYNC JOBS
-- =====
create table if not exists pms_sync_jobs (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    provider_code text not null references pms_providers(code),
    entity_type pms_entity_type not null,
    sync_mode pms_sync_mode not null,
    started_at timestamptz not null default now(),
    finished_at timestamptz null,
    status pms_sync_status not null default 'RUNNING',
    records_read integer not null default 0,
    records_inserted integer not null default 0,
    records_updated integer not null default 0,
    records_failed integer not null default 0,
    error_summary text null,
    created_at timestamptz not null default now()
);
create index if not exists idx_pms_sync_jobs_connection on
pms_sync_jobs(connection_id);
create index if not exists idx_pms_sync_jobs_org on pms_sync_jobs(org_id);
create index if not exists idx_pms_sync_jobs_entity on
pms_sync_jobs(entity_type);
-- =====
-- 7. RAW RECORDS
-- =====
create table if not exists pms_raw_records (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    sync_job_id uuid not null references pms_sync_jobs(id) on delete cascade,
    provider_code text not null references pms_providers(code),
    entity_type pms_entity_type not null,
    external_id text not null,
    external_parent_id text null,
    external_updated_at timestamptz null,
    payload_json jsonb not null,
    payload_hash text not null,
    first_seen_at timestamptz not null default now(),

```

```

    last_seen_at timestamptz not null default now(),
    created_at timestamptz not null default now()
);
create index if not exists idx_pms_raw_records_connection_entity on
pms_raw_records(connection_id, entity_type);
create index if not exists idx_pms_raw_records_external_id on
pms_raw_records(connection_id, external_id);
create index if not exists idx_pms_raw_records_payload_gin on pms_raw_records
using gin (payload_json);
-- =====
-- 8. ROOM TYPES
-- =====
create table if not exists pms_room_types (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid not null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    external_room_type_id text not null,
    code text null,
    name text not null,
    description text null,
    capacity_adults integer null,
    capacity_children integer null,
    max_occupancy integer null,
    is_active boolean not null default true,
    raw_status text null,
    source_updated_at timestamptz null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique (connection_id, external_room_type_id)
);
create index if not exists idx_pms_room_types_org_property on
pms_room_types(org_id, property_id);
create index if not exists idx_pms_room_types_connection on
pms_room_types(connection_id);
drop trigger if exists trg_pms_room_types_updated_at on pms_room_types;
create trigger trg_pms_room_types_updated_at
before update on pms_room_types
for each row execute function set_updated_at();
-- =====
-- 9. ROOMS
-- =====
create table if not exists pms_rooms (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid not null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    external_room_id text not null,
    room_type_id uuid null references pms_room_types(id) on delete set null,
    room_number text null,
    room_name text null,
    floor text null,
    building text null,
    is_active boolean not null default true,
    operational_status pms_room_operational_status not null default 'UNKNOWN',
    raw_status text null,
    source_updated_at timestamptz null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique (connection_id, external_room_id)
);
create index if not exists idx_pms_rooms_org_property on pms_rooms(org_id,
property_id);
create index if not exists idx_pms_rooms_room_type on pms_rooms(room_type_id);

```

```

drop trigger if exists trg_pms_rooms_updated_at on pms_rooms;
create trigger trg_pms_rooms_updated_at
before update on pms_rooms
for each row execute function set_updated_at();
-- =====
-- 10. GUESTS
-- =====
create table if not exists pms_guests (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    external_guest_id text not null,
    external_profile_id text null,
    first_name text null,
    last_name text null,
    full_name text null,
    email text null,
    phone text null,
    document_type text null,
    document_number text null,
    country_code text null,
    nationality_code text null,
    birth_date date null,
    gender text null,
    is_company boolean null,
    company_name text null,
    is_active boolean not null default true,
    source_updated_at timestamptz null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique (connection_id, external_guest_id)
);
create index if not exists idx_pms_guests_org on pms_guests(org_id);
create index if not exists idx_pms_guests_property on pms_guests(property_id);
create index if not exists idx_pms_guests_email on pms_guests(email);
create index if not exists idx_pms_guests_document on
pms_guests(document_number);
drop trigger if exists trg_pms_guests_updated_at on pms_guests;
create trigger trg_pms_guests_updated_at
before update on pms_guests
for each row execute function set_updated_at();
-- =====
-- 11. RESERVATIONS
-- =====
create table if not exists pms_reservations (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_id uuid not null,
    connection_id uuid not null references pms_connections(id) on delete cascade,
    external_reservation_id text not null,
    external_confirmation_code text null,
    external_group_reservation_id text null,
    guest_id uuid null references pms_guests(id) on delete set null,
    primary_room_type_id uuid null references pms_room_types(id) on delete set
null,
    assigned_room_id uuid null references pms_rooms(id) on delete set null,
    status pms_reservation_status not null default 'UNKNOWN',
    raw_status text null,
    booked_at timestamptz null,
    check_in_date date null,
    check_out_date date null,
    nights integer null,
    adults integer null,

```

```

children integer null,
infants integer null,
rooms_count integer null,
channel_code text null,
channel_name text null,
segment_code text null,
segment_name text null,
rate_plan_code text null,
rate_plan_name text null,
currency_code text null,
room_revenue_amount numeric(14,2) null,
total_amount numeric(14,2) null,
paid_amount numeric(14,2) null,
balance_amount numeric(14,2) null,
cancellation_at timestamptz null,
source_updated_at timestamptz null,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),
unique (connection_id, external_reservation_id)
);
create index if not exists idx_pms_reservations_org_property on
pms_reservations(org_id, property_id);
create index if not exists idx_pms_reservations_guest on
pms_reservations(guest_id);
create index if not exists idx_pms_reservations_dates on
pms_reservations(check_in_date, check_out_date);
create index if not exists idx_pms_reservations_status on
pms_reservations(status);
drop trigger if exists trg_pms_reservations_updated_at on pms_reservations;
create trigger trg_pms_reservations_updated_at
before update on pms_reservations
for each row execute function set_updated_at();
-- =====
-- 12. RESERVATION GUESTS
-- =====
create table if not exists pms_reservation_guests (
  id uuid primary key default gen_random_uuid(),
  reservation_id uuid not null references pms_reservations(id) on delete
cascade,
  guest_id uuid not null references pms_guests(id) on delete cascade,
  role pms_guest_role not null default 'UNKNOWN',
  is_main_guest boolean not null default false,
  created_at timestamptz not null default now(),
  unique (reservation_id, guest_id)
);
create index if not exists idx_pms_reservation_guests_reservation on
pms_reservation_guests(reservation_id);
create index if not exists idx_pms_reservation_guests_guest on
pms_reservation_guests(guest_id);
-- =====
-- 13. STAYS
-- =====
create table if not exists pms_stays (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  property_id uuid not null,
  connection_id uuid not null references pms_connections(id) on delete cascade,
  reservation_id uuid null references pms_reservations(id) on delete set null,
  guest_id uuid null references pms_guests(id) on delete set null,
  room_id uuid null references pms_rooms(id) on delete set null,
  room_type_id uuid null references pms_room_types(id) on delete set null,
  external_stay_id text null,
  stay_status pms_stay_status not null default 'UNKNOWN',
  raw_status text null,

```

```

arrival_scheduled_at timestamptz null,
departure_scheduled_at timestamptz null,
check_in_at timestamptz null,
check_out_at timestamptz null,
adults integer null,
children integer null,
currency_code text null,
room_revenue_amount numeric(14,2) null,
total_amount numeric(14,2) null,
source_updated_at timestamptz null,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);
create index if not exists idx_pms_stays_org_property on pms_stays(org_id,
property_id);
create index if not exists idx_pms_stays_reservation on
pms_stays(reservation_id);
create index if not exists idx_pms_stays_guest on pms_stays(guest_id);
create index if not exists idx_pms_stays_room on pms_stays(room_id);
create index if not exists idx_pms_stays_status on pms_stays(stay_status);
drop trigger if exists trg_pms_stays_updated_at on pms_stays;
create trigger trg_pms_stays_updated_at
before update on pms_stays
for each row execute function set_updated_at();
-- =====
-- 14. FOLIO SUMMARIES
-- =====
create table if not exists pms_folio_summaries (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  property_id uuid not null,
  connection_id uuid not null references pms_connections(id) on delete cascade,
  reservation_id uuid null references pms_reservations(id) on delete set null,
  stay_id uuid null references pms_stays(id) on delete set null,
  external_folio_id text null,
  currency_code text null,
  room_charges_amount numeric(14,2) null,
  food_beverage_amount numeric(14,2) null,
  other_charges_amount numeric(14,2) null,
  taxes_amount numeric(14,2) null,
  discounts_amount numeric(14,2) null,
  total_amount numeric(14,2) null,
  balance_amount numeric(14,2) null,
  source_updated_at timestamptz null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
create index if not exists idx_pms_folio_summaries_org_property on
pms_folio_summaries(org_id, property_id);
create index if not exists idx_pms_folio_summaries_reservation on
pms_folio_summaries(reservation_id);
create index if not exists idx_pms_folio_summaries_stay on
pms_folio_summaries(stay_id);
drop trigger if exists trg_pms_folio_summaries_updated_at on
pms_folio_summaries;
create trigger trg_pms_folio_summaries_updated_at
before update on pms_folio_summaries
for each row execute function set_updated_at();
-- =====
-- 15. STATUS MAPPINGS
-- =====
create table if not exists pms_status_mappings (
  id uuid primary key default gen_random_uuid(),
  provider_code text not null references pms_providers(code),

```

```

    entity_type text not null check (entity_type in ('RESERVATION', 'STAY',
'ROOM')),
    raw_value text not null,
    canonical_value text not null,
    is_active boolean not null default true,
    created_at timestamptz not null default now(),
    unique (provider_code, entity_type, raw_value)
);
create index if not exists idx_pms_status_mappings_provider_entity on
pms_status_mappings(provider_code, entity_type);
-- =====
-- 16. CHANNEL MAPPINGS
-- =====
create table if not exists pms_channel_mappings (
    id uuid primary key default gen_random_uuid(),
    provider_code text not null references pms_providers(code),
    raw_channel_code text null,
    raw_channel_name text null,
    canonical_channel text not null,
    created_at timestamptz not null default now()
);
create index if not exists idx_pms_channel_mappings_provider on
pms_channel_mappings(provider_code);
-- =====
-- 17. SEGMENT MAPPINGS
-- =====
create table if not exists pms_segment_mappings (
    id uuid primary key default gen_random_uuid(),
    provider_code text not null references pms_providers(code),
    raw_segment_code text null,
    raw_segment_name text null,
    canonical_segment text not null,
    created_at timestamptz not null default now()
);
create index if not exists idx_pms_segment_mappings_provider on
pms_segment_mappings(provider_code);

```

3) PROMPT MAESTRO PARA CLAUDE CODE

Este prompt sí está pensado para que Claude no improvise arquitectura y trabaje dentro de tus reglas.

ESTAMOS DESARROLLANDO EN DEBACU UNA CAPA UNIVERSAL DE INTEGRACIÓN PMS.
QUIERO QUE ACTÚES COMO ARQUITECTO DE SOFTWARE Y DESARROLLADOR SENIOR FULL-STACK,
CON FOCO EN:

- arquitectura desacoplada
- integración API PMS
- Supabase/Postgres
- React + TypeScript
- Edge Functions
- sistemas multi-tenant
- idempotencia
- seguridad y trazabilidad

CONTEXTO DE NEGOCIO

Debacu es una plataforma SaaS hotelera centrada en:

- riesgo
- revenue intelligence
- analítica operativa

- alertas
 - cuadros de mando multi-propiedad
- Debacu debe integrarse con varios PMS como:
- Tesipro / Ulyses
 - Cloudbeds
 - Mews
 - apaleo
 - Oracle OPERA
 - otros futuros

OBJETIVO ARQUITECTÓNICO

NO queremos construir integraciones específicas que contaminen el core de Debacu.

Queremos un MODELO CANÓNICO UNIVERSAL PMS.

Cada PMS debe mapear sus datos al modelo interno Debacu.

PRINCIPIOS OBLIGATORIOS

1. Debacu define el modelo canónico.
2. Ningún PMS puede imponer su estructura a las tablas core.
3. Toda integración PMS debe guardar:
 - dato raw original
 - dato canonical normalizado
4. Toda entidad PMS debe quedar trazada por:
 - org_id
 - property_id
 - connection_id
5. Fase inicial solo READ-ONLY.
6. El frontend nunca llama directamente al PMS.
7. Toda autenticación PMS se gestiona en backend / Edge Functions.
8. Los conectores deben ser idempotentes.
9. Estados, canales y segmentos deben pasar por mappings explícitos.
10. Debe soportarse multi-property dentro de una misma organización.

MODELO ACTUAL DEFINIDO

Las tablas base del modelo universal PMS son:

INTEGRACIÓN

- pms_providers
- pms_connections
- pms_property_mappings
- pms_sync_jobs
- pms_raw_records

MODELO CANÓNICO

- pms_room_types
- pms_rooms
- pms_guests
- pms_reservations
- pms_reservation_guests
- pms_stays
- pms_folio_summaries

CATÁLOGOS

- pms_status_mappings
- pms_channel_mappings
- pms_segment_mappings

REGLAS DE IMPLEMENTACIÓN

- Usa siempre UUID como PK interna.
- Usa external ids con unicidad compuesta por connection_id.
- No asumas que todos los PMS tienen los mismos campos.
- Mantén campos nulos cuando proceda.
- Separa raw_status de canonical status.
- Separa RESERVATION de STAY aunque algún PMS no lo haga bien.
- Diseña full sync + incremental sync.
- Registra jobs de sincronización y errores.
- Todo conector debe permitir reintento sin duplicados.
- No mezcles UI con lógica de normalización.
- No diseñes pensando en un solo PMS.

CUANDO TE PIDA CÓDIGO QUIERO QUE RESPETES ESTO

1. Si generas SQL:

- que sea compatible con Supabase/Postgres
 - usa nombres claros
 - añade índices razonables
 - evita sobreingeniería innecesaria
2. Si generas TypeScript backend:
 - separa cliente PMS, mapper, sync service y repositorio
 - usa tipos claros
 - añade manejo de errores
 - prepara código idempotente
 3. Si generas frontend:
 - no conectes nunca directo al PMS
 - consume solo APIs internas de Debacu
 4. Si propones arquitectura:
 - explica carpetas
 - explica flujo
 - explica responsabilidades por capa

FORMA DE RESPONDER

Quiero respuestas:

- estructuradas
- técnicas
- realistas
- sin vender humo
- indicando riesgos y decisiones

Si algo depende del PMS y no está confirmado, dilo claramente.

No inventes endpoints de proveedores si no están verificados.

Cuando falte información de un PMS, propón interfaces/adaptadores abstractos y deja marcado qué parte queda pendiente de validación.

PRIMERA TAREA

Usa el modelo universal PMS ya definido para proponer la arquitectura de carpetas y servicios de un primer conector Debacu para Tesipro/Ulyses o, si no hay suficiente documentación pública, para un conector abstracto "PMSBaseConnector" + "TesiproConnector" preparado para completar cuando tengamos la API real.

4) ORDEN REALISTA DE EJECUCIÓN

Para no mezclarlo todo, yo haría esto en este orden:

Paso 1

Crear las tablas SQL del modelo universal PMS.

Paso 2

Crear la estructura backend:

- connectors/
- mappers/
- services/
- repositories/
- types/

Paso 3

Crear un PMSBaseConnector.

Paso 4

Crear un TesiproConnector provisional.

Paso 5

Crear el primer flujo:

- crear connection
- lanzar sync job
- persistir raw
- mapear una entidad sencilla, por ejemplo room_types o reservations

Paso 6

Construir pantalla interna de administración de conexiones PMS